



DATA STRUCTURES

LAB FILE

Submitted by : Himanshu Kumar

Submitted to : SWATI

Course : B.TECH CSE DATA SCIENCE

Semester : III

Roll No : 2401420007

TASK-1

- Store Inventory: To manage items in inventory
 - {Add item, remove item, display inventory, search item}
 - Code:

```
inventory = []

def insert_product():
    sku = input("Enter SKU: ").strip()

    for item in inventory:
        if item['sku'] == sku:
            print("Product with this SKU already exists!")
            return

    name = input("Enter Product Name: ").strip()
    if name == "":
        print("Product name cannot be empty.")
        return

    qty_input = input("Enter Quantity: ")

    if not qty_input.lstrip("-").isdigit():
        print("Invalid quantity.")
        return

    quantity = int(qty_input)

    if quantity < 0:
        print("Quantity must be positive.")
        return

    product = {'sku': sku, 'name': name, 'quantity': quantity}
    inventory.append(product)
    print("Product inserted successfully.")

def display_inventory():
    if not inventory:
        print("Inventory is empty.")
        return

    print("\nCurrent Inventory:")
    print("SKU\t\tProduct Name\t\tQuantity")
    print("-----")
    for item in inventory:
        print(f"{item['sku']}\t\t{item['name']}\t\t{item['quantity']}")
    print()

def search_by_sku():
    sku = input("Enter SKU to search: ").strip()

    for item in inventory:
        if item['sku'] == sku:
```

```

        return
    print("Product not found.")

def search_by_name():
    name = input("Enter Product Name to search: ").strip().lower()
    for item in inventory:
        if item['name'].lower() == name:
            print(f"Product Found: SKU={item['sku']], Name={item['name']], Quantity={item['quantity']}")
            return
    print("Product not found.")

def delete_product():
    sku = input("Enter SKU to delete: ").strip()
    for item in inventory:
        if item['sku'] == sku:
            inventory.remove(item)
            print(f"Product {item['name']} removed from inventory.")
            return
    print("Product not found.")

def main():
    while True:
        print("\nInventory Stock Manager")
        print("1. Insert New Product")
        print("2. Display Inventory")
        print("3. Search Product by SKU")
        print("4. Search Product by Name")
        print("5. Delete Product")
        print("6. Exit")

        choice = input("Enter your choice (1-6): ")

        if choice == '1':
            insert_product()
        elif choice == '2':
            display_inventory()
        elif choice == '3':
            search_by_sku()
        elif choice == '4':
            search_by_name()
        elif choice == '5':
            delete_product()
        elif choice == '6':
            print("Exiting Inventory Manager.")
            break
        else:
            print("Invalid choice. Please select 1 to 6.")

main()

```

*Output:

```

Inventory Stock Manager
1. Insert New Product
2. Display Inventory
3. Search Product by SKU
4. Search Product by Name
5. Delete Product
6. Exit

Enter your choice (1-6): 1
Enter SKU: P101
Enter Product Name: Pencil
Enter Quantity: 50
Product inserted successfully.

```

Enter your choice (1-6): 2

```

Current Inventory:
SKU          Product Name      Quantity
-----
P101         Pencil            50
P102         Pen               20
P103         Pen               20

```

```

1. Insert New Product
2. Display Inventory
3. Search Product by SKU
4. Search Product by Name
5. Delete Product
6. Exit

Enter your choice (1-6): 1
Enter SKU: P102
Enter Product Name: Pen
Enter Quantity: 20
Product inserted successfully.

Inventory Stock Manager
1. Insert New Product
2. Display Inventory
3. Search Product by SKU
4. Search Product by Name
5. Delete Product
6. Exit

```

TASK-2

- Inventory Stock Manager: Reduce stock according to sales and identify zero stock.
 - Code:

```

class Item:
    def __init__(self, item_id, name, stock):
        self.item_id = item_id
        self.name = name
        self.stock = stock

    def display(self):
        print(f"ID: {self.item_id}, Name: {self.name}, Stock: {self.stock}")

class Inventory:
    def __init__(self):
        self.items = []

    def add_item(self, item):
        self.items.append(item)
        print("Item added successfully.")

    def reduce_stock(self, item_id, quantity):
        for item in self.items:
            if item.item_id == item_id:
                if item.stock >= quantity:
                    item.stock -= quantity
                    print(f"Sale successful. Remaining stock: {item.stock}")
                else:
                    print("Not enough stock available!")
                return
        print("Item not found!")
        |
    def zero_stock_items(self):
        print("\n--- Zero Stock Items ---")
        found = False
        for item in self.items:
            if item.stock == 0:
                item.display()
                found = True
        if not found:
            print("No items with zero stock.")

    def display_inventory(self):
        print("\n--- Inventory Stock ---")
        if not self.items:
            print("Inventory is empty.")
        else:
            for item in self.items:
                item.display()

def main():

```



Activate Windows
Go to Settings to activate W

```

inventory = Inventory()

while True:
    print("\n--- Inventory Stock Manager ---")
    print("1. Add Item")
    print("2. Reduce Stock (Sales)")
    print("3. Display Inventory")
    print("4. Identify Zero Stock Items")
    print("5. Exit")

    choice = input("Enter your choice: ")

    if choice == '1':
        item_id = int(input("Enter item ID: "))
        name = input("Enter item name: ")
        stock = int(input("Enter initial stock: "))
        inventory.add_item(Item(item_id, name, stock))

    elif choice == '2':
        item_id = int(input("Enter item ID for sale: "))
        qty = int(input("Enter quantity sold: "))
        inventory.reduce_stock(item_id, qty)

    elif choice == '3':
        inventory.display_inventory()

    elif choice == '4':
        inventory.zero_stock_items()

    elif choice == '5':
        print("Exiting... Thank you!")
        break

    else:
        print("Invalid choice! Please try again.")

if __name__ == "__main__":
    main()

```

Output:

```

--- Inventory Stock Manager ---
1. Add Item
2. Reduce Stock (Sales)
3. Display Inventory
4. Identify Zero Stock Items
5. Exit
Enter your choice: 1
Enter item ID: 9090
Enter item name: Car
Enter initial stock: 100000
Item added successfully.

```

```

--- Inventory Stock Manager ---
1. Add Item
2. Reduce Stock (Sales)
3. Display Inventory
4. Identify Zero Stock Items
5. Exit
Enter your choice: 

```

TASK-3

Inventory system with two functionalities:

1. Calculate Total & Average Stock
2. Find Maximum Stock Item

*CODE:

```

def total_and_avg(inventory):
    if not inventory:
        return 0, 0
    total = sum(qty for _, qty in inventory)
    avg = total / len(inventory)
    return total, avg

def max_stock_item(inventory):
    if not inventory:
        return None, None
    max_item = max(inventory, key=lambda x: x[1])
    return max_item

def main():
    inventory = []

    while True:
        print("\n==== Inventory Menu =====")
        print("1. Add Product to Inventory")
        print("2. Calculate Total and Average Stock")
        print("3. Find Maximum Stock Item")
        print("4. Exit")

        choice = input("Enter your choice (1-4): ")

        if choice == '1':
            sku = int(input("Enter SKU number: "))
            qty = int(input("Enter Quantity: "))
            inventory.append((sku, qty))
            print(f"Product with SKU {sku} and Quantity {qty} added.")

        elif choice == '2':
            total, avg = total_and_avg(inventory)
            print(f"Inventory: {inventory}")
            print(f"Total Stock: {total}")
            print(f"Average Stock: {avg:.2f}")

        elif choice == '3':
            sku, qty = max_stock_item(inventory)
            if sku is None:
                print("Inventory is empty! Add products first.")
            else:
                print(f"Item with Maximum Stock: SKU {sku}, Quantity {qty}")

        elif choice == '4':
            print("Exiting program. Goodbye!")
            break

```

OUTPUT:


```

===== Inventory Menu =====
1. Add Product to Inventory
2. Calculate Total and Average Stock
3. Find Maximum Stock Item
4. Exit
Enter your choice (1-4): 1
Enter SKU number: 101
Enter Quantity: 20
Product with SKU 101 and Quantity 20 added.

===== Inventory Menu =====
1. Add Product to Inventory
2. Calculate Total and Average Stock
3. Find Maximum Stock Item
4. Exit
Enter your choice (1-4): 1
Enter SKU number: 103
Enter Quantity: 30
Product with SKU 103 and Quantity 30 added.

===== Inventory Menu =====
1. Add Product to Inventory
2. Calculate Total and Average Stock
3. Find Maximum Stock Item
4. Exit
Enter your choice (1-4): 2
Inventory: [(101, 20), (103, 30)]
Total Stock: 50
Average Stock: 25.00

===== Inventory Menu =====
1. Add Product to Inventory
2. Calculate Total and Average Stock
3. Find Maximum Stock Item
4. Exit
Enter your choice (1-4): 3
Item with Maximum Stock: SKU 103, Quantity 30

===== Inventory Menu =====
1. Add Product to Inventory
2. Calculate Total and Average Stock
3. Find Maximum Stock Item
4. Exit
Enter your choice (1-4): 4
Exiting program. Goodbye!

```

TASK 4: STACK OPERATIONS: {push,pop,peek,display}

CODE:

```
stack = []

def push(item):
    stack.append(item)
    print("Item pushed.")

def pop_item():
    if not stack:
        print("Stack is empty!")
    else:
        print("Popped:", stack.pop())

def peek():
    if not stack:
        print("Stack is empty!")
    else:
        print("Top element:", stack[-1])

def display():
    if not stack:
        print("Stack is empty!")
    else:
        print("Stack elements:", stack)

push(10)
push(20)
push(30)

display()
peek()
pop_item()
display()
```

Output:

```
Item pushed.
Item pushed.
Item pushed.
Stack elements: [10, 20, 30]
Top element: 30
Popped: 30
Stack elements: [10, 20]
```

TASK-5: Implementation of stack browser:

Code:

```

back_stack = []
forward_stack = []
current_page = None

def visit(page):
    global current_page
    if current_page:
        back_stack.append(current_page)
    current_page = page
    forward_stack.clear()
    print("Visited:", page)

def back():
    global current_page
    if not back_stack:
        print("No previous page!")
    else:
        forward_stack.append(current_page)
        current_page = back_stack.pop()
        print("Back to:", current_page)

def forward():
    global current_page
    if not forward_stack:
        print("No forward page!")
    else:
        back_stack.append(current_page)
        current_page = forward_stack.pop()
        print("Forward to:", current_page)

def show():
    print("Current Page:", current_page)

visit("google.com")
visit("youtube.com")
visit("github.com")

back()
back()
forward()
show()

```

Output:

```

Visited: google.com
Visited: youtube.com
Visited: github.com
Back to: youtube.com
Back to: google.com
Forward to: youtube.com
Current Page: youtube.com

```

TASK-6: Insertion in circular linkedlist

CODE:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class CircularLinkedList:
    def __init__(self):
        self.head = None

    def insert_at_end(self, data):
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
            new_node.next = self.head
            return
        temp = self.head
        while temp.next != self.head:
            temp = temp.next
        temp.next = new_node
        new_node.next = self.head

    def insert_at_beginning(self, data):
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
            new_node.next = self.head
            return
        temp = self.head
        while temp.next != self.head:
            temp = temp.next
        new_node.next = self.head
        temp.next = new_node
        self.head = new_node

    def insert_at_middle(self, pos, data):
        if pos == 1:
            self.insert_at_beginning(data)
            return
        new_node = Node(data)
        temp = self.head
        count = 1
        while count < pos - 1 and temp.next != self.head:
            temp = temp.next
            count += 1
        new_node.next = temp.next
        temp.next = new_node
```

```

def display(self):
    if self.head is None:
        print("List is empty")
        return
    temp = self.head
    while True:
        print(temp.data, end=" -> ")
        temp = temp.next
        if temp == self.head:
            break
    print("(back to head)")

c11 = CircularLinkedList()
c11.insert_at_end(10)
c11.insert_at_end(20)
c11.insert_at_end(30)

print("Original list:")
c11.display()

c11.insert_at_beginning(5)
print("\nAfter inserting 5 at beginning:")
c11.display()

c11.insert_at_end(40)
print("\nAfter inserting 40 at end:")
c11.display()

c11.insert_at_middle(3, 15)
print("\nAfter inserting 15 at position 3:")
c11.display()

```

OUTPUT:

```

Original list:
10 -> 20 -> 30 -> (back to head)

After inserting 5 at beginning:
5 -> 10 -> 20 -> 30 -> (back to head)

After inserting 40 at end:
5 -> 10 -> 20 -> 30 -> 40 -> (back to head)

After inserting 15 at position 3:
5 -> 10 -> 15 -> 20 -> 30 -> 40 -> (back to head)

```

TASK-7:

CODE:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class CircularLinkedList:
    def __init__(self):
        self.head = None

    def insert(self, data):
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
            self.head.next = self.head
        else:
            temp = self.head
            while temp.next != self.head:
                temp = temp.next
            temp.next = new_node
            new_node.next = self.head

    def display(self):
        if self.head is None:
            print("List is empty")
            return
        temp = self.head
        print("Circular Linked List:", end=" ")
        while True:
            print(temp.data, end=" -> ")
            temp = temp.next
            if temp == self.head:
                break
        print("(back to head)")

    def delete_begin(self):
        if self.head is None:
            print("List is empty, nothing to delete.")
            return
        if self.head.next == self.head:
            self.head = None
            print("Deleted the only node in the list.")
            return
        last = self.head
        while last.next != self.head:
            last = last.next
        last.next = self.head.next
        self.head = self.head.next
        print("Node deleted from beginning.")
```

```

class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class CircularLinkedList:
    def __init__(self):
        self.head = None

    def insert(self, data):
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
            self.head.next = self.head
        else:
            temp = self.head
            while temp.next != self.head:
                temp = temp.next
            temp.next = new_node
            new_node.next = self.head

    def display(self):
        if self.head is None:
            print("List is empty")
            return
        temp = self.head
        print("Circular Linked List:", end=" ")
        while True:
            print(temp.data, end=" -> ")
            temp = temp.next
            if temp == self.head:
                break
        print("(back to head)")

    def delete_begin(self):
        if self.head is None:
            print("List is empty, nothing to delete.")
            return
        if self.head.next == self.head:
            self.head = None
            print("Deleted the only node in the list.")
            return
        last = self.head
        while last.next != self.head:
            last = last.next
        last.next = self.head.next
        self.head = self.head.next
        print("Node deleted from beginning.")

```

```

        if self.head.next == self.head:
            self.head = None
            print("Deleted the only node in the list.")
            return
        prev = None
        temp = self.head
        while temp.next != self.head:
            prev = temp
            temp = temp.next
        prev.next = self.head
        print("Node deleted from end.")

    def delete_middle(self, pos):
        if self.head is None:
            print("List is empty, nothing to delete.")
            return
        if pos == 1:
            self.delete_begin()
            return
        temp = self.head
        count = 1
        prev = None
        while count < pos and temp.next != self.head:
            prev = temp
            temp = temp.next
            count += 1
        if count < pos:
            print("Position out of range.")
            return
        prev.next = temp.next
        print("Node deleted from position", pos)

c11 = CircularLinkedList()
c11.insert(10)
c11.insert(20)
c11.insert(30)
c11.insert(40)

print("Initial List:")
c11.display()

c11.delete_begin()
c11.display()

c11.delete_end()
c11.display()

c11.delete_middle(2)
c11.display()

```


OUTPUT:

```
Initial List:  
Circular Linked List: 10 -> 20 -> 30 -> 40 -> (back to head)  
Node deleted from beginning.  
Circular Linked List: 20 -> 30 -> 40 -> (back to head)  
Node deleted from end.  
Circular Linked List: 20 -> 30 -> (back to head)  
Node deleted from position 2  
Circular Linked List: 20 -> (back to head)
```

TASK 8: Circular queue insertion and deletion

CODE:

```
MAX_SIZE = 100

class CircularQueue:
    def __init__(self):
        self.front = -1
        self.rear = -1
        self.arr = [0] * MAX_SIZE

    def isFull(self):
        if (self.front == 0 and self.rear == MAX_SIZE - 1) or (self.rear == (self.front - 1) % (MAX_SIZE - 1)):
            return True
        return False

    def isEmpty(self):
        return self.front == -1

    def enqueue(self, value):
        if self.isFull():
            print("Queue is full.")
        else:
            if self.front == -1:
                self.front = 0
            self.rear = (self.rear + 1) % MAX_SIZE
            self.arr[self.rear] = value
            print("Enqueued element:", value)

    def dequeue(self):
        if self.isEmpty():
            print("Queue is empty.")
            return -1
        element = self.arr[self.front]
        if self.front == self.rear:
            self.front = -1
            self.rear = -1
        else:
            self.front = (self.front + 1) % MAX_SIZE
        print("Dequeued element:", element)
        return element

    def display(self):
        if self.isEmpty():
            print("Queue is empty.")
        else:
            print("Elements in the queue:", end=" ")
            i = self.front
            while i != self.rear:
                print(self.arr[i], end=" ")
                i = (i + 1) % MAX_SIZE
```

```

        i = (i + 1) % MAX_SIZE
        print(self.arr[i])

q = CircularQueue()
q.enqueue(10)
q.enqueue(20)
q.enqueue(30)
q.enqueue(40)

q.display()

q.dequeue()
q.dequeue()

q.display()

q.enqueue(50)
q.enqueue(60)

q.display()

```

OUTPUT:

```

Enqueued element: 10
Enqueued element: 20
Enqueued element: 30
Enqueued element: 40
Elements in the queue: 10 20 30 40
Dequeued element: 10
Dequeued element: 20
Elements in the queue: 30 40
Enqueued element: 50
Enqueued element: 60
Elements in the queue: 30 40 50 60

```

TASK-9:

LINEAR SEARCH

CODE:

```
import random

arr = [random.randint(1, 100) for _ in range(10)]
print("Array:", arr)

key = int(input("Enter element to search: "))

found = False
for i in range(len(arr)):
    if arr[i] == key:
        print("Element found at index", i)
        found = True
        break

if not found:
    print("Element not found")
```

OUTPUT:

```
Array: [54, 90, 11, 46, 11, 77, 100, 97, 10, 22]
Enter element to search: 46
Element found at index 3
```

TASK-10:

BINARY SEARCH:

CODE:

```
import random

arr = sorted([random.randint(1, 100) for _ in range(10)])
print("Array:", arr)

key = int(input("Enter element to search: "))

low = 0
high = len(arr) - 1
found = False

while low <= high:
    mid = (low + high) // 2
    if arr[mid] == key:
        print("Element found at index", mid)
        found = True
        break
    elif arr[mid] < key:
        low = mid + 1
    else:
        high = mid - 1

if not found:
    print("Element not found")
```

OUTPUT:

```
Array: [6, 26, 42, 46, 61, 65, 66, 67, 68, 89]
Enter element to search: 61
Element found at index 4
```

TASK 11:BUBBLE SORT

CODE:

```
import random

arr = [random.randint(1, 100) for _ in range(10)]
print("Original array:", arr)

n = len(arr)
for i in range(n):
    for j in range(0, n - i - 1):
        if arr[j] > arr[j + 1]:
            arr[j], arr[j + 1] = arr[j + 1], arr[j]

print("Sorted array:", arr)
```

OUTPUT:

```
Original array: [39, 52, 68, 28, 28, 72, 3, 28, 70, 69]
Sorted array: [3, 28, 28, 28, 39, 52, 68, 69, 70, 72]
```